

Data types

Data type defines the kind of value a variable holds (number, text, decimal, true/false, etc.) and how much memory it uses.

How to use:

Declare type before variable name

- int age = 21; double marks = 89.5;

When to use:

Pick based on the kind of value:

- whole number → int,
- decimal → double
- Character → char
- true/false → boolean
- text → string

Where to use:

Every variable declaration, method parameters, return types, arrays.

Why Specific type matters:

Different types use different memory (int = 4 bytes, double = 8 bytes); prevents wasted memory and logical errors; only number types support math operations.

Two Categories of data types

Primitive

int(4), double(8), float(4),
byte(1), short(2), char(1), } bytes
boolean(1) } bit

} stores simple values directly

Non-Primitive (reference)

Strings, Arrays, }
classes, }
Interfaces }

} store references to objects, more complex

Why 8 primitive types (not one universal type):

Memory efficiency - pick the smallest type that fits the data.

Why long needs "L":

Whole number literals default to "int"; "L" tells Java to treat it as long before overflow happens.

Why double is default over float:

More precision (64 bit vs 32-bit); most calculations need that accuracy.

Why String isn't primitive:

Text length varies, but primitives are fixed size. So, String is an object that manages a dynamic character array internally.

Syntax Pattern:

```
int rollNo = 45;  
double cgpa = 8.75;  
char section = 'A';  
boolean isPresent = true;  
String branch = "CSE";
```

Real Life Example:

Like different containers for different ingredients - a jar for sugar, a bottle for oil. Each type "Container" suits a specific kind of data.

- We can't store flour into bottle (or) we can't store oil in jar, etc...

- So, we have specific types to store data in java.